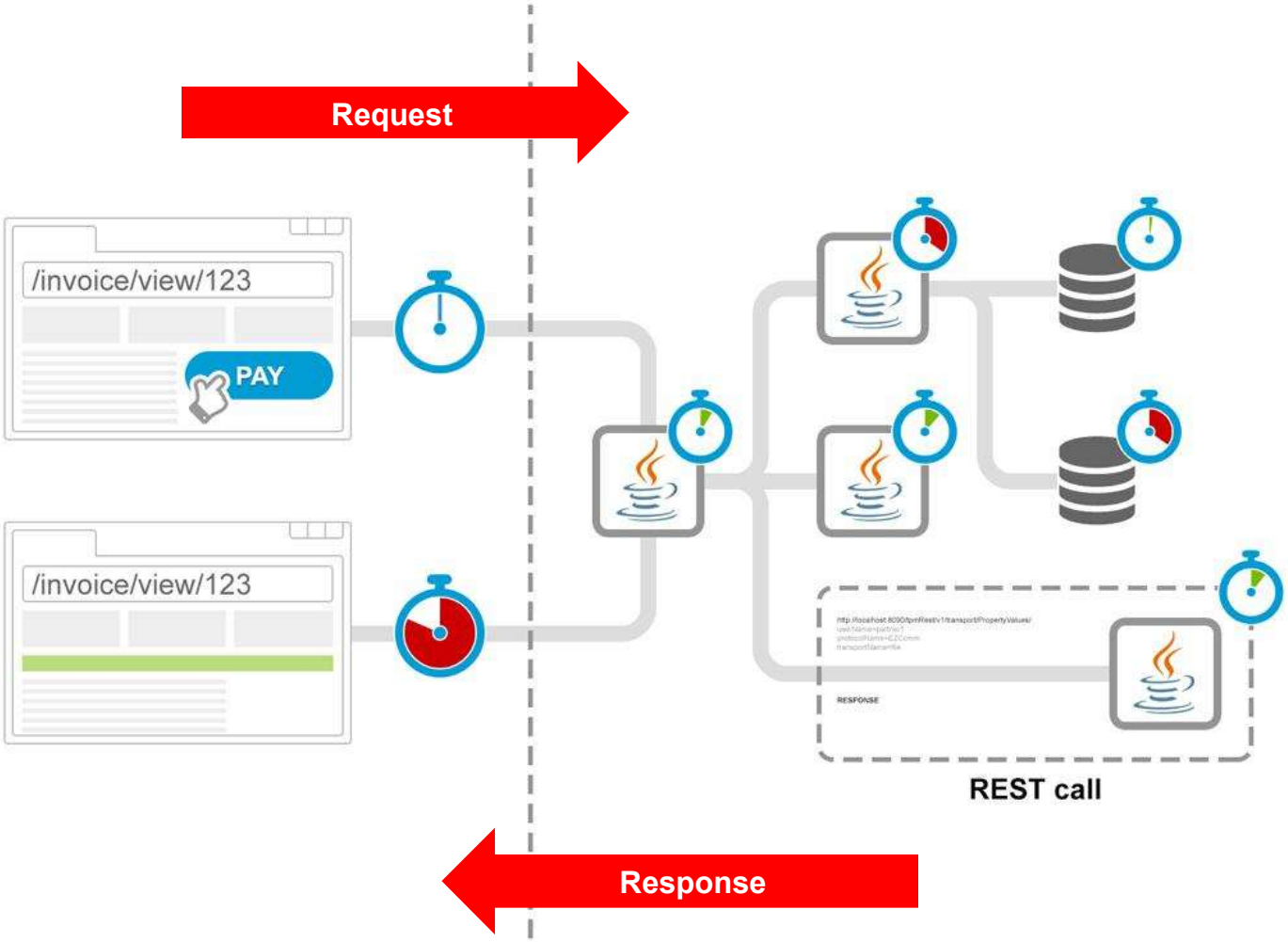
React Fundamentals
Module – Http communication

Peter Kassenaar –
info@kassenaar.com

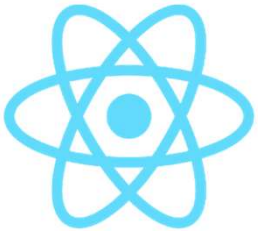
Communicating with external APIs

Fetching and handling external data over http

Calling a backend from your React App

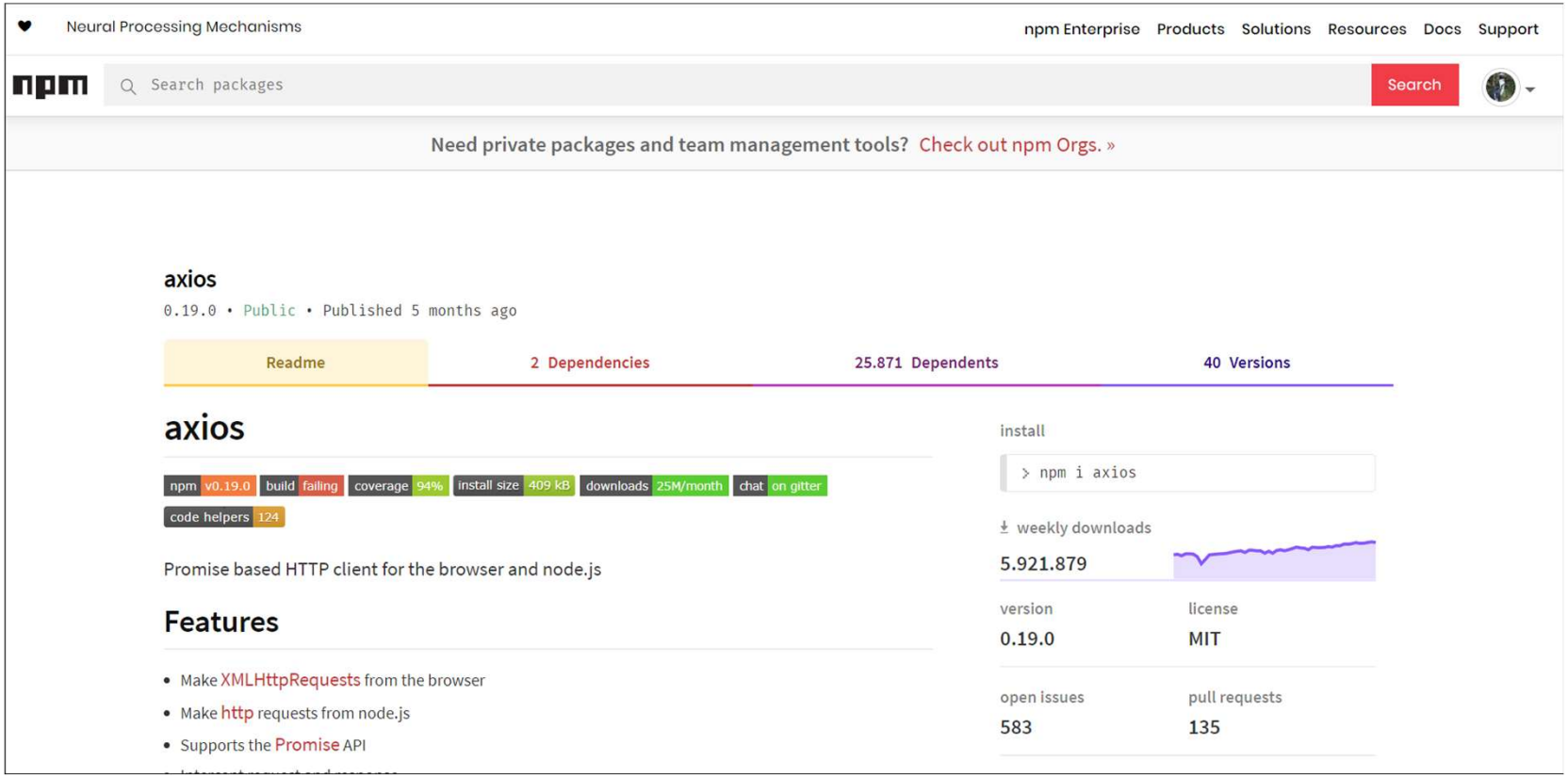


Options for using external data



- Tons of external (open/public) API's available
  - https://github.com/public-apis/public-apis
- React can *not* do http-calls by itself.
- Options for calling them from your app
  - `fetch` – natively built in to the browser
  - JQuery `$.ajax()`
  - `axios` – popular open source http-library
  - …lots of other options

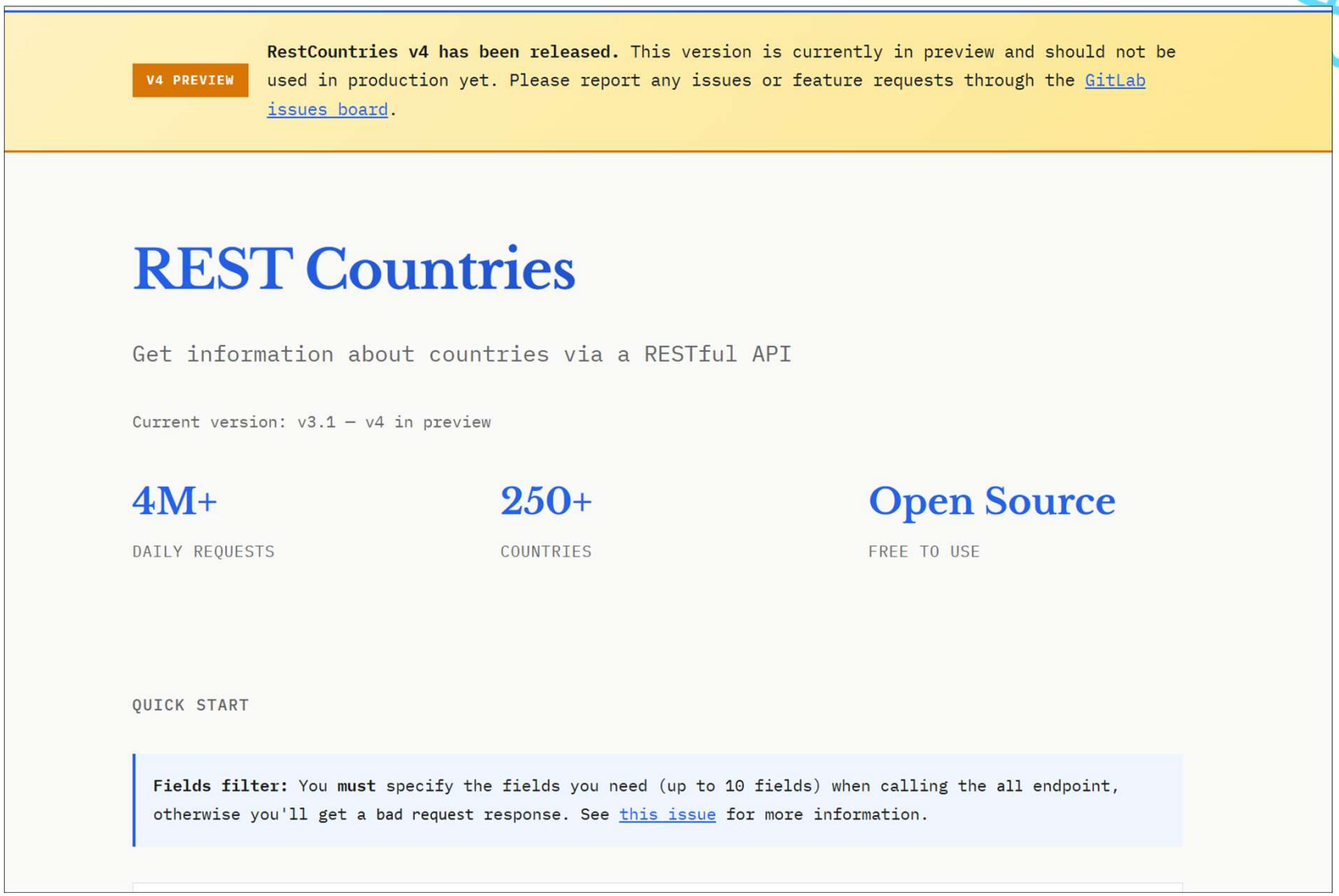
Our choice: `axios`




```
npm install axios [--save]
```

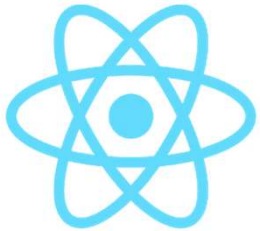
https://www.npmjs.com/package/axios

API of choice: Restful Countries



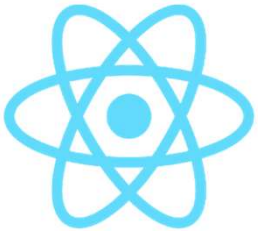
https://restcountries.com/

Goal – fetching country data from API



1. Prepare State for the data you're about to receive
2. Discover the URL that you want to talk to
3. Use `axios` to call the url
   - For instance in the `componentDidMount()` lifecycle hook
   - But it can be in a button `clickHandler` as well..
4. Set the state in the `.then()` handler of the promise
5. Pass state to the component as usual

Set state and call URL



```
import axios from 'axios'
const url = 'https://restcountries.com/v3.1/all?...';
```

URL to talk to

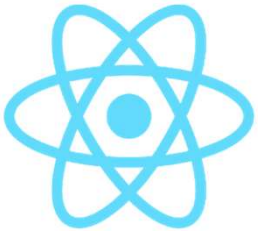
```
const [error, setError] = useState(null);
const [isLoaded, setIsLoaded] = useState(false);
const [countries, setCountries] = useState([]);
```

Prepare state

```
useEffect(() => {
    // using setTimeout here, to emulate network delay.
    setTimeout(() => {
        axios.get(url)
            .then(response => {
                setIsLoaded(true);
                setCountries(response.data); // JSON
            })
            .catch(error => {
                console.log('Error!! :: ' + error)
                setError(error);
                setIsLoaded(true);
            })
    }, 1000)
}, []);
```

Set state with data

Loading indicator, aka Spinner

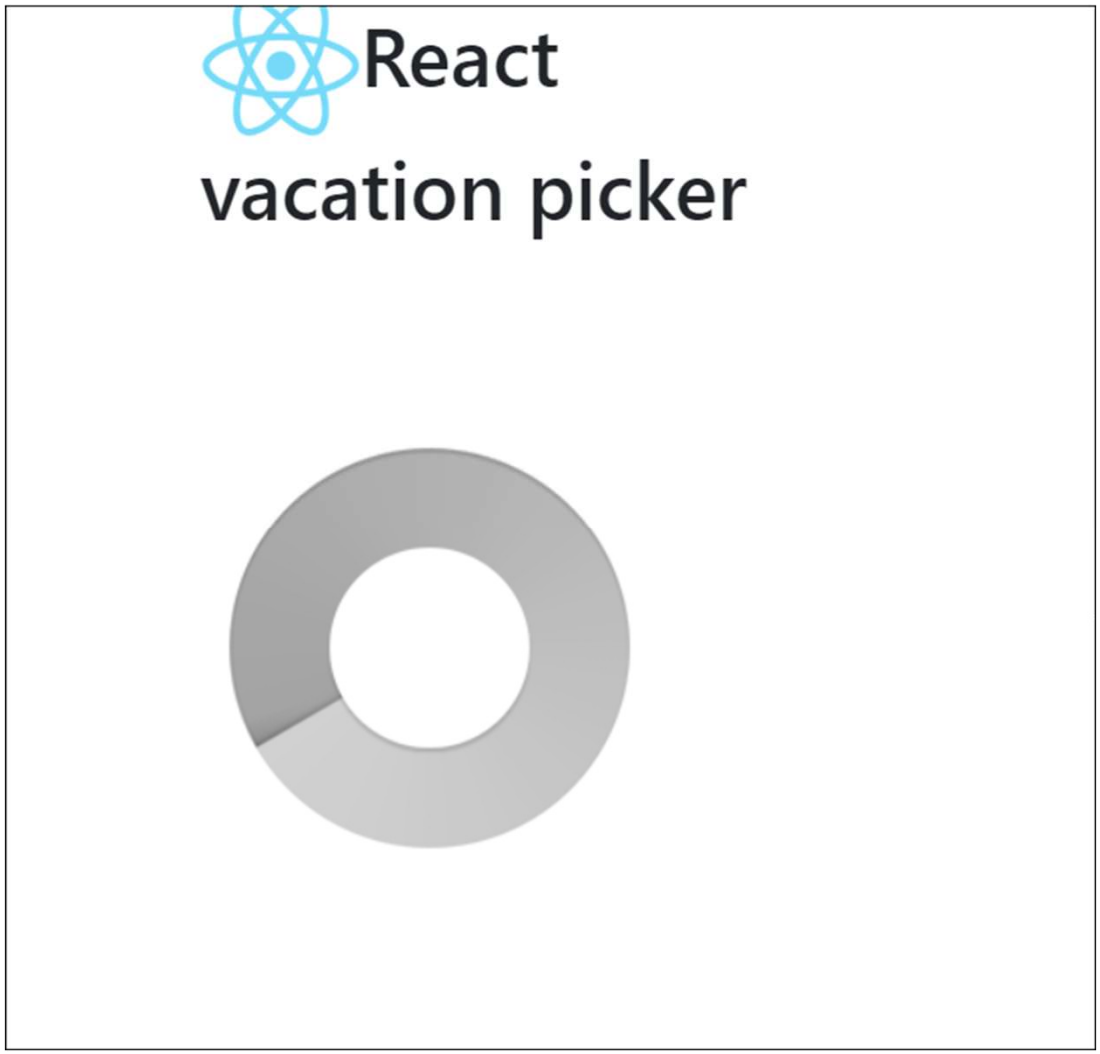


Depends on `isLoaded`. For instance like:

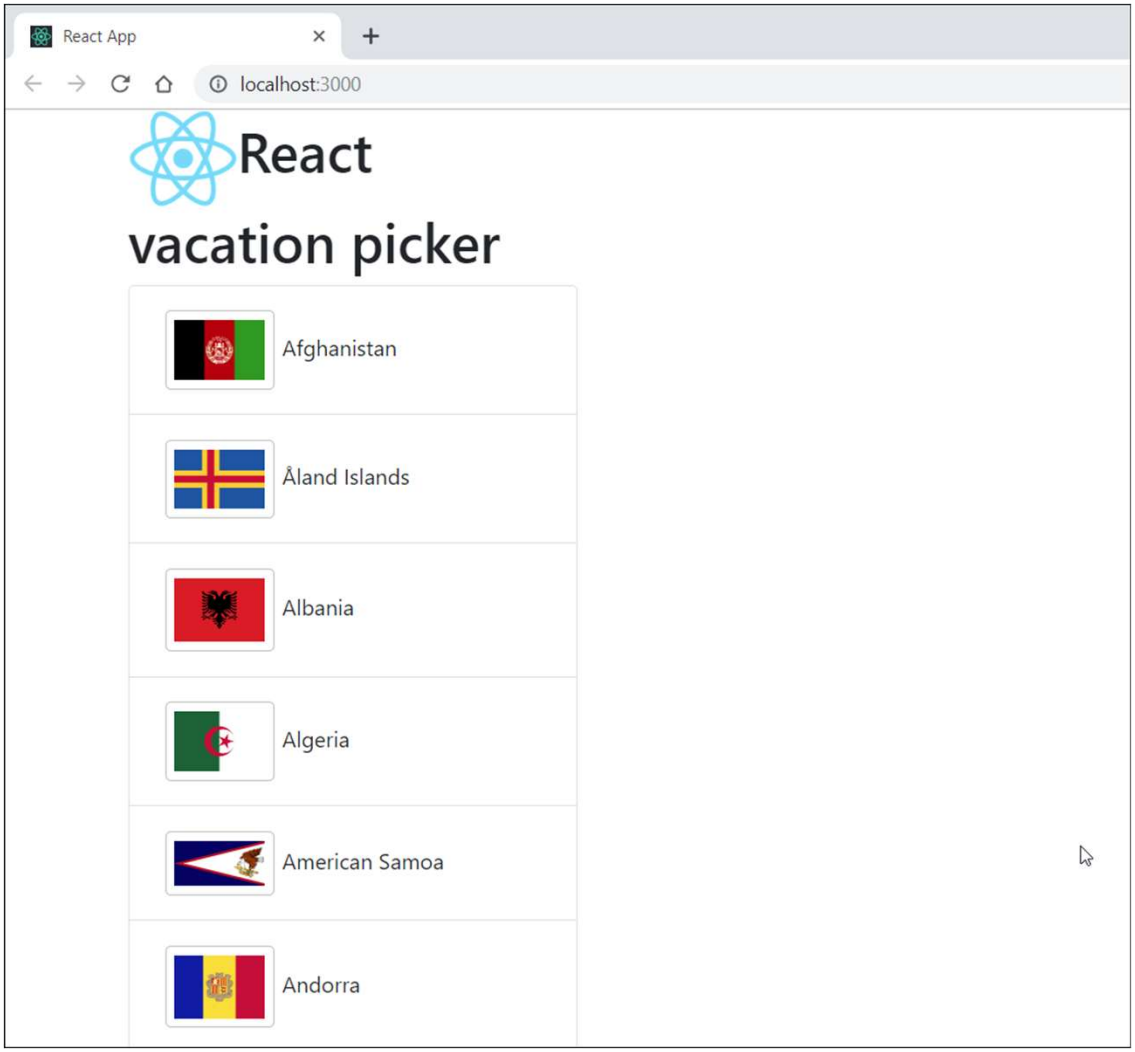
```
{/*Show a loading indicator as long as the countries are not loaded*/}
{
    !this.state.isLoaded &&
       <LoadingIndicator/>
}
```

```
const LoadingIndicator = () => {
    return (
        <div>
            <img src={require('./spinner.gif')} alt="Please wait..."/>
        </div>
    );
}
```

Result w/ simulated delay



Result after delay

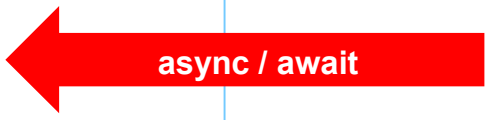


../examples/500-api-call

On async / await

- You *can* use the `async/await` notation if you like.
  - It's just syntactic sugar over Promises

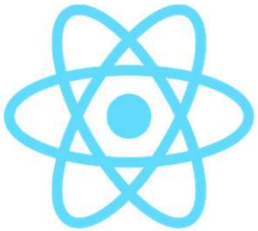
```
useEffect(() => {
    // using setTimeout here, to emulate network delay.
    setTimeout(async () => {
        const response = await axios.get(url);
        setIsLoaded(true)
        setCountries(response.data)
    }, 1000)
}, []);
```



Fetching details

Doing subsequent API calls

Same principle

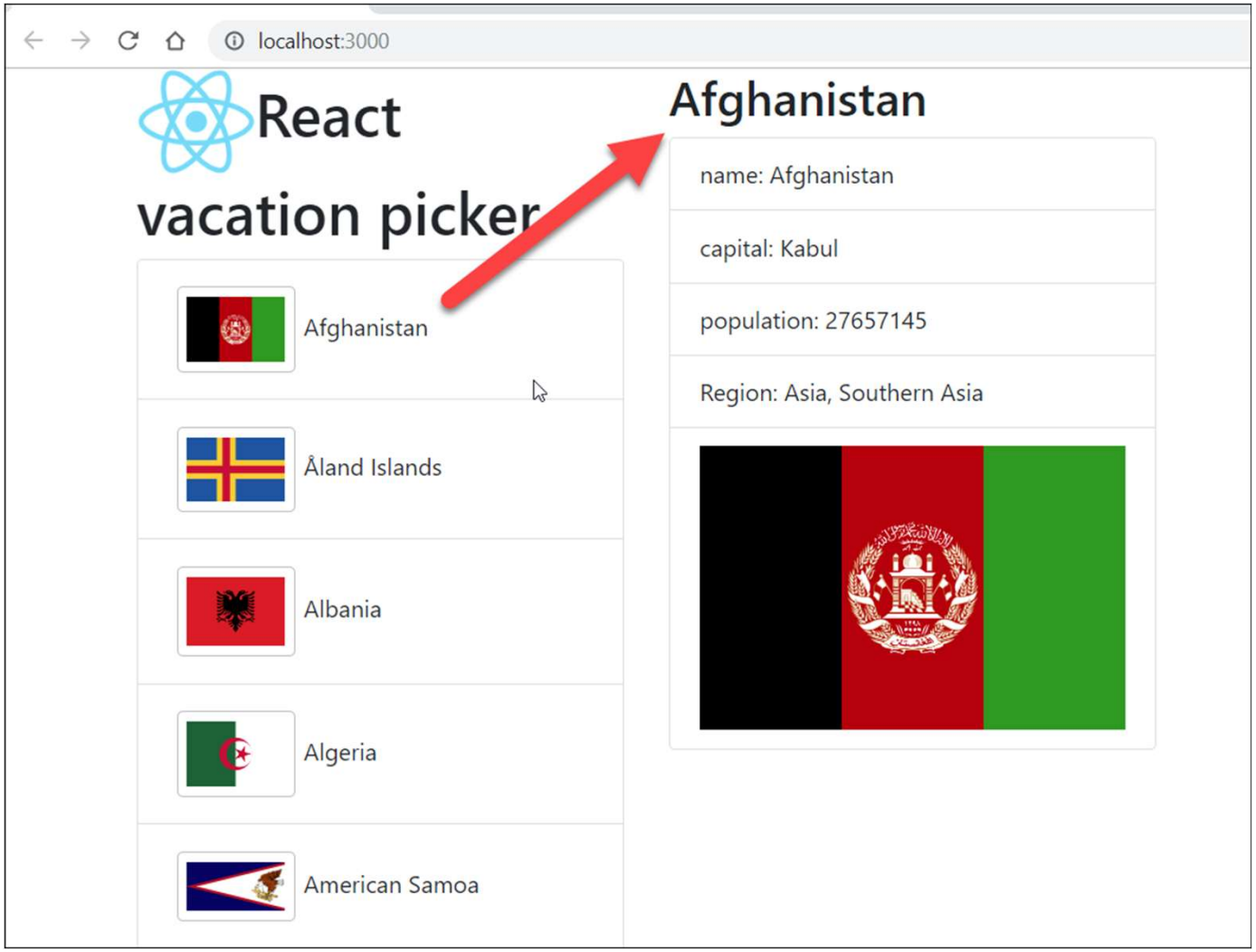


- Pass click handler as prop to `<VacationPicker />`
- find URL, do an `axios` call, update state
- Pass the state to a (new) detail component

```
<VacationPicker
    countries={countries}
    select={(country) => getCountry(country)}
/>
```

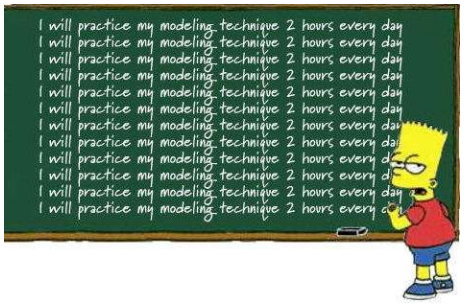
```
// get details for a specific country
getCountry(name) {
    axios.get(`${detail_url}/${name}`)
        .then(response => {
            setCountry(response.data[0]);
        })
}
```

Result

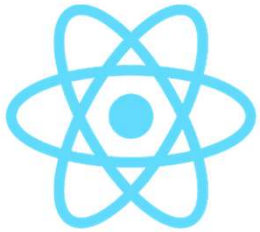

localhost:3000
React
vacation picker
Afghanistan
Åland Islands
Albania
Algeria
American Samoa
Afghanistan
name: Afghanistan
capital: Kabul
population: 27657145
Region: Asia, Southern Asia

Workshop

- Create a new project, or work from the example project
  - Install `axios` if necessary
- Use the **Typicode API** to fetch dummy random users:
  - https://jsonplaceholder.typicode.com/users
- Show usernames and e-mail addresses in a list
- Clicking a user fetches details for that specific user. Show them in a detail component
- Optional – Create a `<SearchCountry>` component for the example app
  - where the user can type in (a part of) the name.
  - Search for that country/countries, show them.
- Example `../500-api-call`



More API's



VacationPicker.js | JavaScript APIs.txt

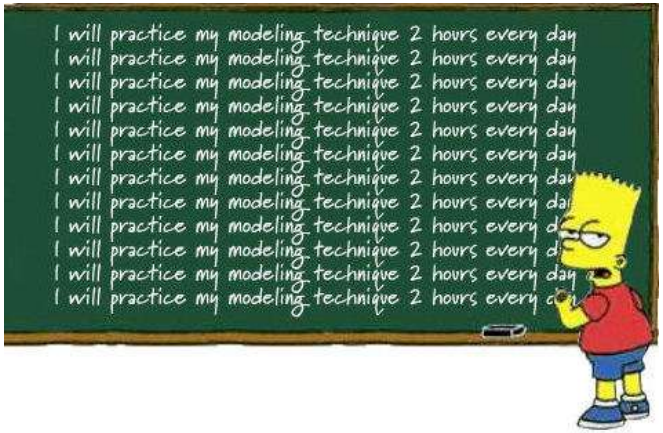
```
// DEZE API's zijn open, zonder registratie beschikbaar, of al geregistreerd (reg
// THESE API's are open, free, mostly available without registration (or already

https://opendata.rdw.nl/resource/m9d7-ebf2.json?kenteken= + kenteken
http://swapi.co/ - The Star Wars API
http://api.openweathermap.org/data/2.5/weather?units=metric&appid=8566d604cd9402b
http://ergast.com/mrd/ - Ergast Motor (Formule 1) API
https://randomuser.me/api/?results=10 - random user data
http://www.omdbapi.com/?apikey=f1f56c8e& - Open Movie Database (gebruik liever je
https://api.github.com/users/ - GitHub user information.
http://dev.markitondemand.com/Api/v2/Quote/json?symbol=AAPL  - Aandelen/stock quo
http://restcountries.eu/ - Information on all countries in the world.
https://restcountries.eu/rest/v1/all - Andere API met Country-informatie
http://api.postcodedata.nl/v1/postcode/?postcode=1211EP&streetnumber=60&ref=domei
http://opendata.cbs.nl/dataportaal/portal.html?_la=nl&_catalog=CBS - Honderden AP
https://data.pdok.nl/datasets - Allerlei informatie van het Kadaster
https://coinmarketcap.com/api/ - Blockchain/Bitcoin information
https://www.anwb.nl/feeds/gethf - realtime file-informatie Nederlandse snelwegen
https://api.spacexdata.com/v3/launches/ - SpaceX rocket launch information
```

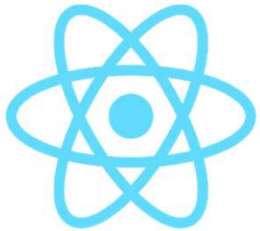
File: `JavaScript APIs.txt` in the root of the repository

Workshop #2

- Create a small application using one of the other API's in the file `JavaScript API's.txt`
- Store and fetch data in the state. Use for instance:
  - Pokemon API
  - Open Movie Database API
  - OpenWeatherMap API
  - …



Checkpoint



- You know how to make Ajax/API calls from an application
- You are able to handle data coming from a Restful endpoint and show it in your app
- ALWAYS read the documentation of the corresponding API